

Licenciatura en Ingeniaría de Ciberseguridad e Infraestructura de Cómputo

Programación en Red

Protocolo HTTP

Brayan Usciel Seseña Hidalgo

Práctica 7: Capturar y leer tráfico HTTP real

Parte 1

Ejemplo de tráfico en la pestaña de red

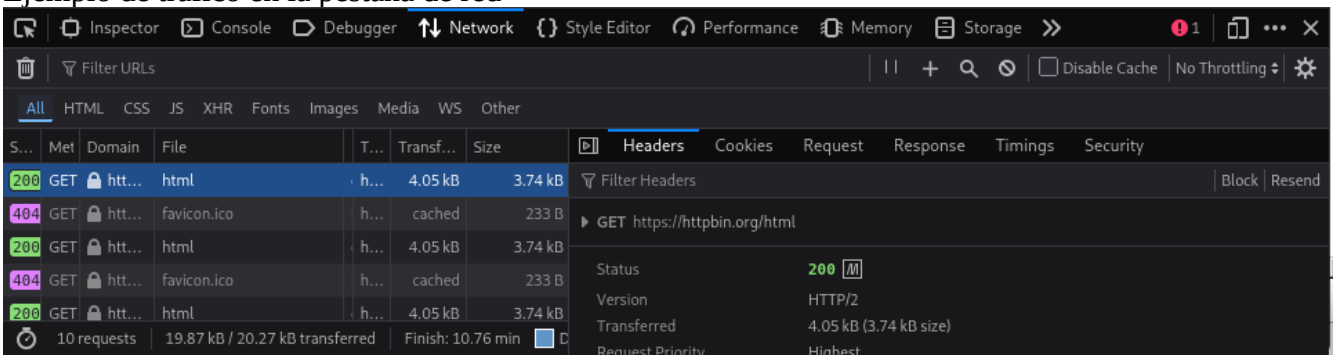


Figura 1. Tráfico HTTP del sitio https://httpbin.org/html

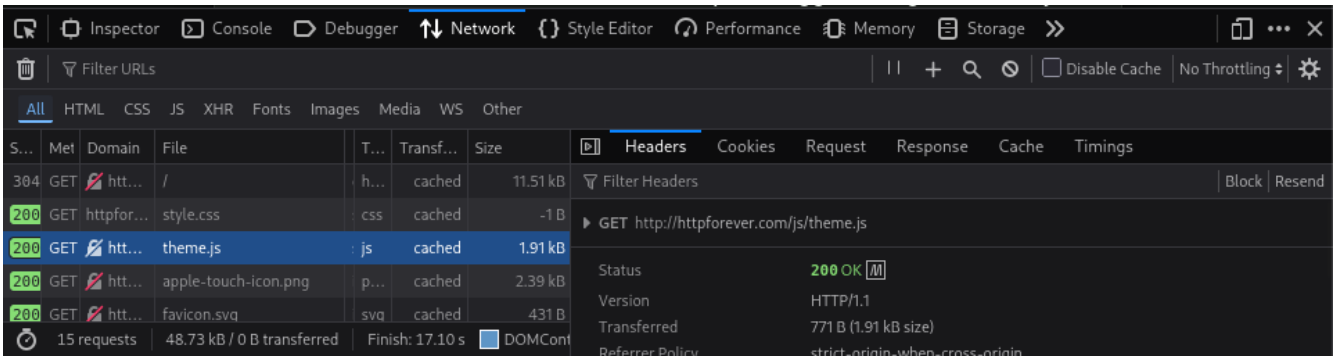


Figura 2. Tráfico HTTP del sitio http://httpforever.com/

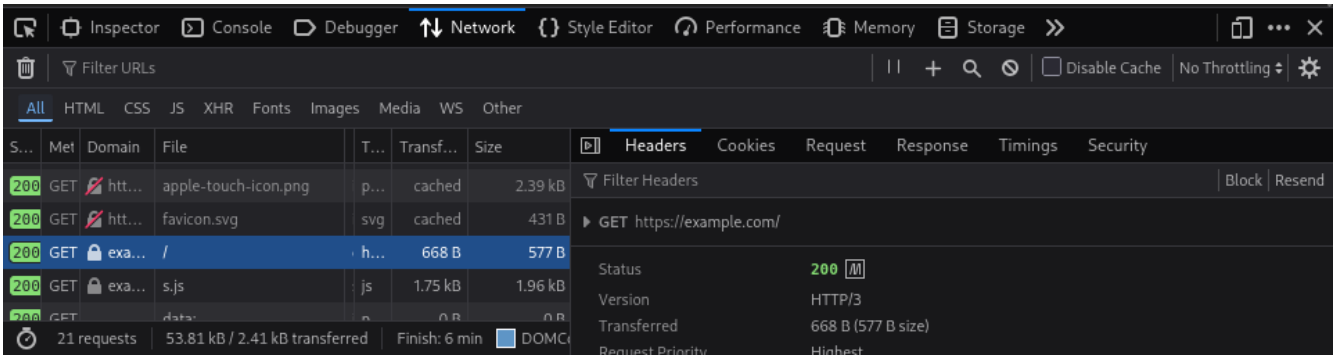


Figura 3. Tráfico HTTP del sitio https://example.com/

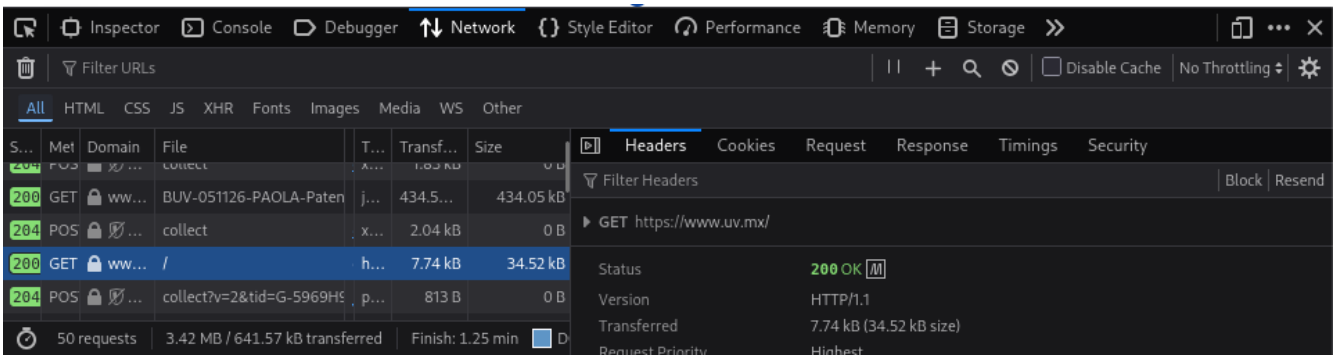


Figura 4. Tráfico HTTP del sitio https://www.uv.mx/

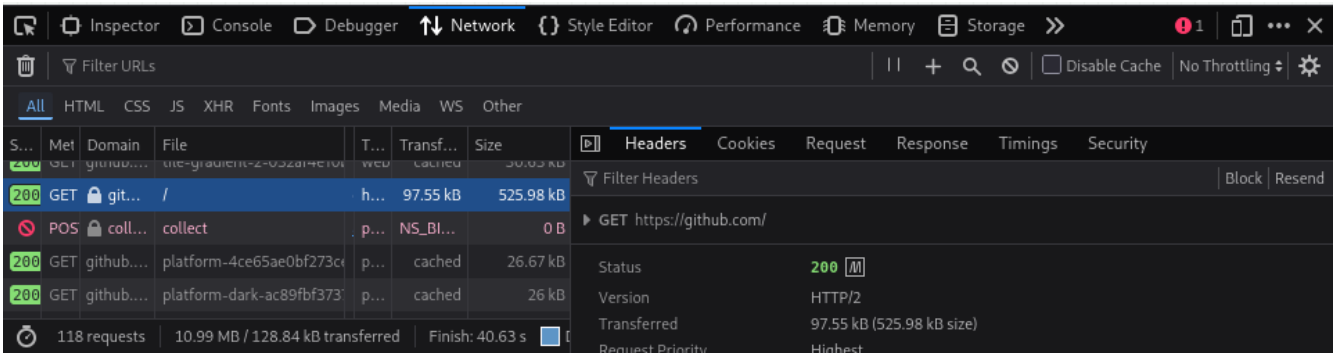


Figura 5. Tráfico HTTP del sitio https://github.com/

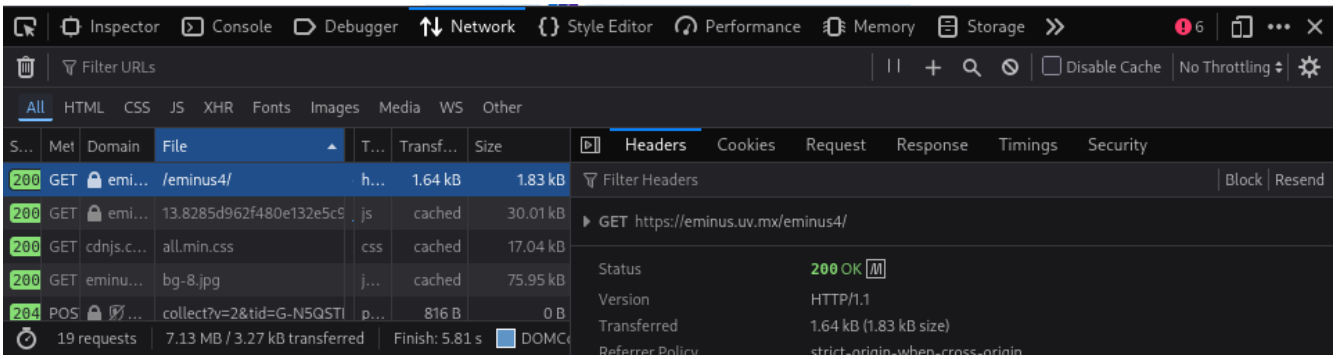


Figura 6. Tráfico HTTP del sitio https://eminus.uv.mx/eminus4

Tabla de registros

4. En el panel de detalle (Headers / Encabezados) localicen y anoten: la línea de solicitud (método, ruta, versión), la línea de estado (versión, código, frase) y el User-Agent con el que se presentó su navegador.

Sitio	Línea de solicitud	Línea de estado	User-Agent
https:// httpbin.org/html	Método: GET https://httpbin.org/html Versión: HTTP/2	Versión: HTTP/2 Código: 200 Frase: OK	Mozilla/5.0 (X11; Linux x86_64; rv:153.0) Gecko/20100101 Firefox/153.0

	Se hace una solicitud GET para acceder a un recurso.		
httpforever.com	Metodo: GET http://httpforever.com/ Versión: HTTP/1.1 Se hace una solicitud GET para acceder al recurso principal.	Versión: HTTP/1.1 Código: 200 Frese: OK	Mozilla/5.0 (X11; Linux x86_64; rv:153.0) Gecko/20100101 Firefox/153.0
example.com	Método GET https://example.com/ Versión: HTTP/3 Se hace una solicitud de tipo GET.	Versión: HTTP/3 Código: OK Frase: OK	Mozilla/5.0 (X11; Linux x86_64; rv:153.0) Gecko/20100101 Firefox/153.0
uv.mx	Método GET https://www.uv.mx/ Versión: HTTP/1.1 Hace una solicitud de tipo GET	Versión HTTP/1.1 Código: 200 Frase: OK	Mozilla/5.0 (X11; Linux x86_64; rv:153.0) Gecko/20100101 Firefox/153.0
github.com	Meódo GET https://github.com/ Versión: HTTP/2 Realiza una solicitud GET	Versión HTTP/2 Código: 200 Frase: OK	Mozilla/5.0 (X11; Linux x86_64; rv:153.0) Gecko/20100101 Firefox/153.0
eminus.uv.mx/ eminus4/	Método GET https://eminus.uv.mx/ eminus4 Versión: HTTP/1.1 Hace una solicitud GET para obtener el recurso	Versión HTTP/1.1 Código: 200 Frase: OK	Mozilla/5.0 (X11; Linux x86_64; rv:153.0) Gecko/20100101 Firefox/153.0

Contestar

• ¿Por qué el encabezado Host es obligatorio en HTTP/1.1 si la conexión ya sabe a qué IP llegó?

El encabezado Host es obligatorio en HTTP/1.1 principalmente alojamiento virtual basado en nombres. Permite que un mismo servidor (con una única dirección IP y puerto) aloje cientos de sitios web o dominios diferentes.

• Si HTTP es texto plano y legible, ¿qué implica eso para las contraseñas que viajan en un formulario? (anticipo de la clase de HTTPS)
Pueden ser susceptibles a interceptaciones, como ataques Man-In-The-Middle, ya que con algún sniffer como Wireshark es posible ver el tráfico de paquetes en la red, en consecuencia seria facil obtener credenciales de un sitio HTTP ya que no esta cifrado.

• ¿Qué tienen en común el protocolo apodo>mensaje de la Unidad I, el XML-RPC de la semana pasada y HTTP?
Que devuelven una respuesta particular dada la solicitud de un cliente, por ejemplo con el protocolo de la práctica apodo>mensaje se reenviaba el mensaje y el apodo del cliente desde el servidor, en XML-RPC un usuario consultaba funciones y dichas funciones le respondían de forma particular a cada usuario, de la misma forma HTTP, responde a las solicitudes GET particulares de cada usuario.

Parte 2

Ejercicio 1 · Un 200 a mano
• Con curl -v pidan https://httpbin.org/get. Identifiquen en la salida: la línea de solicitud que ustedes generaron,
el User-Agent de curl y la línea de estado 200.

```
✔ [09:09 ] lonerism@loretta:~ 192.168.100.4 79% $ curl -v https://httpbin.org/ge
t
* Host httpbin.org:443 was resolved.
* IPv6: 2600:1f18:2da1:d402:54c2:29a8:4a48:2547, 2600:1f18:2da1:d400:5c03:f07e:e
0be:7edd, 2600:1f18:2da1:d401:3bc4:1b7d:6e3d:8acb
* IPv4: 3.221.253.123, 44.209.237.184, 54.172.45.38
*   Trying [2600:1f18:2da1:d402:54c2:29a8:4a48:2547]:443...
* ALPN: curl offers h2,http/1.1
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
*  CAfile: /etc/ssl/certs/ca-certificates.crt
*  CApath: /etc/ssl/certs
* TLSv1.3 (IN), TLS handshake, Server hello (2):
* TLSv1.3 (IN), TLS change cipher, Change cipher spec (1):
* TLSv1.3 (IN), TLS handshake, Encrypted Extensions (8):
* TLSv1.3 (IN), TLS handshake, Unknown (25):
* TLSv1.3 (IN), TLS handshake, CERT verify (15):
* TLSv1.3 (IN), TLS handshake, Finished (20):
* TLSv1.3 (OUT), TLS change cipher, Change cipher spec (1):
* TLSv1.3 (OUT), TLS handshake, Finished (20):
* SSL connection using TLSv1.3 / TLS_AES_256_GCM_SHA384 / x25519 / id-ecPublicKe
y
* ALPN: server accepted h2
* Server certificate:
*  subject: CN=httpbin.hbin02.postmanlabs.com
```

Para esta consulta el user-agent fue el propio curl y el código de respuesta fue 200.

```
> User-Agent: curl/8.14.1
> Accept: */*
>
* Request completely sent off
* TLSv1.3 (IN), TLS handshake,
* TLSv1.3 (IN), TLS handshake,
< HTTP/2 200
```

Ejercicio 2 · Un 404 provocado

• Pidan una ruta que no existe: curl -v https://httpbin.org/noexiste. Registren la línea de estado y expliquen por qué es un error 4xx y no 5xx.

```
✔ [09:14 ] lonerism@loretta:~ 192.168.100.4 79% $ curl -v https://httpbin.org/no
existe.
* Host httpbin.org:443 was resolved.
* IPv6: 2600:1f18:2da1:d400:5c03:f07e:e0be:7edd, 2600:1f18:2da1:d402:54c2:29a8:4
a48:2547, 2600:1f18:2da1:d401:3bc4:1b7d:6e3d:8acb
* IPv4: 54.172.45.38, 3.221.253.123, 44.209.237.184
*   Trying [2600:1f18:2da1:d400:5c03:f07e:e0be:7edd]:443...
* ALPN: curl offers h2,http/1.1
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
*  CAfile: /etc/ssl/certs/ca-certificates.crt
```

```
> User-Agent: curl/8.14.1
> Accept: */*
>
* Request completely sent o
* TLSv1.3 (IN), TLS handsha
* TLSv1.3 (IN), TLS handsha
< HTTP/2 404
```

Se trata de un error 400 ya que el error es por parte del cliente a intentar obtener un recurso que no existe, en cambio, si fuera un error 500 seria un error por parte del servidor.

Ejercicio 3 · Un 500 bajo pedido

• Pidan https://httpbin.org/status/500: el servidor simula una falla interna. Registren la respuesta y expliquen la diferencia de responsabilidad frente al 404.

```
✔ [09:14 ] lonerism@loretta:~ 192.168.100.4 79% $ curl -v https://httpbin.org/st
atus/500
* Host httpbin.org:443 was resolved.
* IPv6: 2600:1f18:2da1:d402:54c2:29a8:4a48:2547, 2600:1f18:2da1:d400:5c03:f07e:e
0be:7edd, 2600:1f18:2da1:d401:3bc4:1b7d:6e3d:8acb
* IPv4: 44.209.237.184, 54.172.45.38, 3.221.253.123
*   Trying [2600:1f18:2da1:d402:54c2:29a8:4a48:2547]:443...
```

```
> GET /status/500 HTTP/2
> Host: httpbin.org
> User-Agent: curl/8.14.1
> Accept: */*
>
* Request completely sent o
* TLSv1.3 (IN), TLS handsha
* TLSv1.3 (IN), TLS handsha
< HTTP/2 500
```

En este error 500 simulado, suponiendo que es uno verdadero, la responsabilidad es totalmente del servidor, ya que el error 500 suele ser el error común al haber algún error interno del servidor, ya sea que no esta disponible en ese momento, que no pudo conectarse a otro servidor, etc. Es diferente al error 404 anterior ya que ahí el error es debido al usuario al consultar un recurso que no existe.

Ejercicio 4 · Métodos distintos de GET

• Envíen un POST con cuerpo a https://httpbin.org/post (como en el ejemplo de la sección 5) y un HEAD a https://httpbin.org/get (curl -I). Anoten qué cambió en la respuesta de cada uno.

```
✔ [09:24 ] lonerism@loretta:~ 192.168.100.4 79% $ curl -X POST -d "Hola Mundo" h
ttps://httpbin.org/post
{
  "args": {},
  "headers": {
    "Accept": "*/*",
    "Content-Length": "10",
    "Content-Type": "application/x-www-form-urlencoded",
    "Host": "httpbin.org",
    "User-Agent": "curl/8.14.1"
  },
  "method": "POST",
  "origin": "2806:2f0:7580:f371:a3d6:b69f:6bb8:4795",
  "url": "https://httpbin.org/post",
  "data": "",
  "files": {},
  "form": {
    "Hola Mundo": ""
  },
  "json": null
}
```

```
✔ [09:24 ] lonerism@loretta:~ 192.168.100.4 79% $ curl -I https://httpbin.org/ge
t
HTTP/2 200
access-control-allow-credentials: true
access-control-allow-origin: *
access-control-expose-headers: Retry-After, RateLimit, RateLimit-Policy, WWW-Aut
henticate
content-length: 227
content-type: application/json
x-content-type-options: nosniff
date: Sat, 14 Nov 2026 03:25:28 GMT
```

Para el envió de POST, se pretende realizar cambios en el servidor, incluso parece que intenta modificar un archivo JSON para registrar la carga de información. Mientras que en HEAD no se pretende alterar el servidor sino solo consultar los encabezados de un recurso.

Ejercicio 5 · La petición literal con netcat

Escriban la petición con printf + nc como en la sección 5. Comparen: ¿qué encabezados agrega curl por su cuenta que ustedes no escribieron a mano?

```
✔ [09:25 ] lonerism@loretta:~ 192.168.100.4 79% $ printf "GET /get HTTP/1.1\\r\\
nHost: httpbin.org\\r\\n\\r\\n" | nc httpbin.org 80
HTTP/1.1 200 OK
access-control-allow-credentials: true
access-control-allow-origin: *
access-control-expose-headers: Retry-After, RateLimit, RateLimit-Policy, WWW-Aut
henticate
content-length: 148
content-type: application/json
x-content-type-options: nosniff
date: Sat, 14 Nov 2026 03:35:08 GMT

{
  "args": {},
  "headers": {
    "Host": "httpbin.org"
  },
  "method": "GET",
  "origin": "187.190.156.159",
  "url": "http://httpbin.org/get"
}
```

Se agregan encabezados como los content type y length, la fecha de la consulta en el horario del servidor, y lo que parece ser una dirección IP, aunque esa dirección no esta asociada a su dominio de DNS, ya que al hacer una consulta con dig al dominio esa dirección IP no aparece, por lo que puede ser alguna otra dirección IP asociada al servidor del sitio.

```
✔ [09:39 ] lonerism@loretta:~ 192.168.100.4 79% $ dig httpbin.org

; <<>> DiG 9.20.30-1~deb13u1-Debian <<>> httpbin.org
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 14028
;; flags: qr rd ra; QUERY: 1, ANSWER: 3, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 512
;; QUESTION SECTION:
;httpbin.org.                   IN      A

;; ANSWER SECTION:
httpbin.org.            38      IN      A       54.172.45.38
httpbin.org.            38      IN      A       44.209.237.184
httpbin.org.            38      IN      A       3.221.253.123

;; Query time: 79 msec
;; SERVER: 100.100.100.100#53(100.100.100.100) (UDP)
;; WHEN: Fri Nov 13 21:43:24 CST 2026
;; MSG SIZE  rcvd: 88
```